

UNIVERSIDAD TECNOLÓGICA METROPOLITANA (UTM)

Documentación Técnica Oficial: POO, SOLID, Patrones y Memoria HEAP Sistema de Gestión 'Salón de Belleza Nicté Ha' — Examen Ordinario

Este documento detalla de forma breve, clara y rigurosa la ubicación de código (inicio a fin), fragmento de sintaxis, **CÓMO** fue implementado y **POR QUÉ** se utilizó cada concepto en el proyecto ASP.NET Core MVC.

1. HERENCIA

Ubicación del Archivo:	Models/Cliente.cs y Models/Estilista.cs
Rango de Líneas (Inicio a Fin):	Cliente.cs: Líneas 1 a 15 Estilista.cs: Líneas 1 a 15
Código / Sintaxis:	<pre>public class Cliente : Persona { public string Telefono { get; set; } } public class Estilista : Persona { public string NumeroEmpleado { get; set; } }</pre>
¿CÓMO se aplicó?:	Las clases Cliente y Estilista heredan mediante el operador ':' de la clase abstracta base Persona.
¿POR QUÉ se usó?:	Reutiliza atributos comunes (Id, Nombre, Correo) sin duplicar código entre roles del salón.

2. ABSTRACCIÓN

Ubicación del Archivo:	Models/Persona.cs e Patrones/Observer/CitaObserver.cs
Rango de Líneas (Inicio a Fin):	Persona.cs: Líneas 1 a 14 CitaObserver.cs: Líneas 14 a 17
Código / Sintaxis:	<pre>public abstract class Persona { public int Id { get; set; } public string Nombre { get; set; } } public interface IObservadorCita { void Actualizar(int citaId, string nuevoEstado, string detalle); }</pre>
¿CÓMO se aplicó?:	Define la clase abstracta Persona e interfaces puras como IObservadorCita.
¿POR QUÉ se usó?:	Aísla los detalles concretos y expone únicamente los contratos esenciales del negocio.

3. ENCAPSULAMIENTO

Ubicación del Archivo:	Models/Cita.cs y Patrones/Singleton/BackupManager.cs
Rango de Líneas (Inicio a Fin):	Cita.cs: Líneas 12, 32 a 37 BackupManager.cs: Líneas 13 a 15
Código / Sintaxis:	<pre>public DateTime FechaHoraFin { get; private set; } public void AsignarHorario(DateTime inicio) { FechaHoraInicio = inicio; FechaHoraFin = inicio.AddHours(1); }</pre>
¿CÓMO se aplicó?:	Utiliza modificador 'private set' en la propiedad y calcula la hora fin mediante el método AsignarHorario().
¿POR QUÉ se usó?:	Protege el estado interno del objeto e impide modificar manualmente la duración fija de 1 hora por cita.

4. POLIMORFISMO

Ubicación del Archivo:	Patrones/Factory/NotificacionFactory.cs y Patrones/Observer/CitaObserver.cs
Rango de Líneas (Inicio a Fin):	NotificacionFactory.cs: Líneas 12 a 29 CitaObserver.cs: Líneas 23 a 51
Código / Sintaxis:	<pre>public class NotificacionSms : INotificacion { public void Enviar(string msg) { Console.WriteLine('SMS: ' + msg); } }</pre>
¿CÓMO se aplicó?:	Múltiples clases implementan el mismo método Enviar() o Actualizar() reaccionando según su tipo concreto.
¿POR QUÉ se usó?:	Permite invocar notificaciones de manera uniforme sin importar la vía física de transmisión.

5. CONSTRUCTOR Y DESTRUCTOR

Ubicación del Archivo:	Models/Cita.cs y Patrones/Singleton/BackupManager.cs
Rango de Líneas (Inicio a Fin):	Cita.cs: Líneas 25 a 28 BackupManager.cs: Línea 15
Código / Sintaxis:	<pre>public Cita() { Estado = 'Agendada'; } private BackupManager() { }</pre>
¿CÓMO se aplicó?:	Constructores públicos para inicializar valores por defecto y constructor privado en Singletons.
¿POR QUÉ se usó?:	Garantiza un estado inicial válido al instanciar citas. La destrucción la administra el Garbage Collector de .NET.

6. INSTANCIA (Instanciación)

Ubicación del Archivo:	Controllers/CitasController.cs y AdminController.cs
Rango de Líneas (Inicio a Fin):	CitasController.cs: Líneas 44 a 48 AdminController.cs: Línea 80
Código / Sintaxis:	<pre>var nuevaCita = new Cita { ClienteId = clienteId, ServicioId = servicioId };</pre>
¿CÓMO se aplicó?:	Materialización concreta de clases mediante el operador 'new'.
¿POR QUÉ se usó?:	Crea físicamente los registros de citas solicitados por los clientes durante la ejecución.

7. HEAP (Memoria Administrada)

Ubicación del Archivo:	Controllers/AdminController.cs y Data/SalonDbContext.cs
Rango de Líneas (Inicio a Fin):	AdminController.cs: Líneas 34 a 45
Código / Sintaxis:	<pre>var citasList = _context.Citas.ToList(); var sujeto = new SujetoCita();</pre>
¿CÓMO se aplicó?:	Todos los objetos de tipo referencia (listas, contexto EF Core, modelos) se almacenan en la memoria HEAP del CLR.
¿POR QUÉ se usó?:	Permite preservar estructuras complejas vivas durante el ciclo de vida de las peticiones HTTP.

8. PRINCIPIOS SOLID

Ubicación del Archivo:	Controllers/AdminController.cs y Patrones/
-------------------------------	--

Rango de Líneas (Inicio a Fin):	AdminController.cs: Líneas 1 a 20 (Encabezado descriptivo)
Código / Sintaxis:	<pre>// SRP: BackupManager solo hace respaldos. // OCP: SujetoCita permite nuevos observadores sin modificar su código. // LSP: Cliente sustituye a Persona. // ISP: IObservadorCita es una interfaz delgada. // DIP: AdminController usa Inyección de Dependencias.</pre>
¿CÓMO se aplicó?:	Aplicación de los 5 principios de diseño en arquitectura de clases e inyección de dependencias.
¿POR QUÉ se usó?:	Garantiza un sistema desacoplado, fácil de probar, extensible y de alta mantenibilidad según rúbrica UTM.

9. PATRÓN SINGLETON

Ubicación del Archivo:	Patrones/Singleton/BackupManager.cs y ConfiguracionSalon.cs
Rango de Líneas (Inicio a Fin):	BackupManager.cs: Líneas 1 a 45 ConfiguracionSalon.cs: Líneas 1 a 37
Código / Sintaxis:	<pre>public class BackupManager { private static BackupManager _instancia; private static readonly object _lock = new object(); private BackupManager() { } public static BackupManager Instancia { get { lock (_lock) { if (_instancia == null) _instancia = new BackupManager(); return _instancia; } } } }</pre>
¿CÓMO se aplicó?:	Constructor privado, variable estática y acceso mediante propiedad pública 'Instancia' con cerrojo de hilo.
¿POR QUÉ se usó?:	Garantiza una única instancia centralizada en toda la aplicación para generar respaldos de BD.

10. PATRÓN FACTORY METHOD

Ubicación del Archivo:	Patrones/Factory/NotificacionFactory.cs
Rango de Líneas (Inicio a Fin):	NotificacionFactory.cs: Líneas 35 a 53
Código / Sintaxis:	<pre>public static INotificacion FabricarNotificacion(string tipo) { if (tipo == 'SMS') return new NotificacionSms(); return new NotificacionCorreo(); }</pre>
¿CÓMO se aplicó?:	Método de fabricación estático que evalúa parámetros y retorna objetos concretos que implementan INotificacion.
¿POR QUÉ se usó?:	Desacopla al creador de las clases concretas de notificación para recordatorios de citas.

11. PATRÓN OBSERVER: 3 OBSERVADORES CONCRETOS Y SUJETO

Ubicación del Archivo:	Patrones/Observer/CitaObserver.cs
-------------------------------	-----------------------------------

Rango de Líneas (Inicio a Fin):	Observador 1 (Sms): Líneas 23 a 29 Observador 2 (Correo): Líneas 34 a 40 Observador 3 (Bitácora): Líneas 45 a 51 Sujeto (SujetoCita): Líneas 56 a 80
Código / Sintaxis:	<pre> // Observador 1: public class ObservadorSmsCliente : IObservadorCita { public void Actualizar(int id, string est, string det) { Console.WriteLine(\$"[SMS CLIENTE]: Cita #{id} -> '{est}'. {det}"); } } // Observador 2: public class ObservadorCorreoCliente : IObservadorCita { public void Actualizar(int id, string est, string det) { Console.WriteLine(\$"[CORREO CLIENTE]: Cita #{id} -> '{est}'. {det}"); } } // Observador 3: public class ObservadorBitacoraAdmin : IObservadorCita { public void Actualizar(int id, string est, string det) { Console.WriteLine(\$"[BITÁCORA ADMIN]: Auditoría Cita #{id} -> '{est}'. "); } } // Sujeto Publicador: public class SujetoCita { private readonly List _obs = new List(); public void Suscribirse(IObservadorCita o) { _obs.Add(o); } public void Notificar(int id, string est, string det) { foreach(var o in _obs) o.Actualizar(id, est, det); } } </pre>
¿CÓMO se aplicó?:	El SujetoCita mantiene una lista de suscriptores y los notifica automáticamente en bucle al cambiar el estado de una cita.
¿POR QUÉ se usó?:	1. ObservadorSmsCliente: Envía alerta SMS al cliente. 2. ObservadorCorreoCliente: Envía confirmación por correo. 3. ObservadorBitacoraAdmin: Registra evento en la auditoría del administrador. Permite notificación multicanal desacoplada (Principio OCP).